

الكود — مرجع المناقشة

كل رقم سطر هنا حقيقي. لو قالولك «وريني»، تفتح الملف وتروح للسطر على طول.

cicd-failure-prediction/src/ — ٦ ملفات بس هي اللي تهتم.

١. الخريطة — الشغل ماشي إزاي

```
collect_github_data.py → data/raw/github_actions_real.csv (9,772 rows) ↓
data_preparation.py → data/processed/cicd_prepared.csv ↓ (cleaning · engineered features ·
text cleaning · stoplist · splits) hybrid_pipeline.py → ColumnTransformer + 3 classifiers
↓ cross_validation.py → commit-grouped 5-fold + threshold selection ↓ train_evaluate.py →
metrics · ablation · business model ↓ run_corrected_evaluation.py → كل رقم وكل
فصل ٧
```

أمر واحد يعيد كل حاجة: `./reproduce.sh` (٣~ دقائق)

٢. `collect_github_data.py` — ٥٥٠ سطر

مسؤول عن: الكلام مع GitHub API وإنتاج الـ CSV الخام. مشغولش تاني بعد كده.

السطر	الدالة	بتعمل إيه
124	<code>build_session(token)</code>	auth headers بال requests session
138	<code>_maybe_sleep_for_rate_limit()</code>	بيقرا <code>X-RateLimit-Remaining</code> وينام لو قَرَّب يخلص
157	<code>github_get()</code>	GET بـ retries و exponential backoff
239	<code>iter_workflow_runs()</code>	pagination على <code>/actions/runs</code>
267	<code>fetch_commit()</code>	call تانية لـ <code>/commits/{sha}</code> علشان الـ diff stats
292	<code>normalise_row()</code>	بيحوّل الـ JSON لصف مسطّح في الـ CSV
342	<code>load_existing_run_ids()</code>	resume — ميجيبش اللي جابه قبل كده

أسئلة متوقعة

«ليه call اتنين لكل run؟»

«ال workflow runs endpoint بيدّيني الـ run والنتيجة بتاعته، بس مبيدّينيش أي حاجة عن الـ commit نفسه. عدد السطور والملفات المتغيّرة موجودين في الـ commits endpoint بس. فمحتاج الاتنين — وده اللي خلّي الجمع ياخذ ساعتين لـ ٩٧٧٢ run.»

«إزاي اتعاملت مع الـ rate limit؟»

«الدالة في سطر ١٣٨ بتقرا `X-RateLimit-Remaining` من كل response، ولو قَرَّب يخلص بتنام لحد ما الـ window يتجدد. وفيه retries بـ exponential backoff للأخطاء المؤقتة. والدالة في سطر ٣٤٢ بتخلّي العملية resumable — لو وقفت في النص، أعيد تشغيلها وهي بتكمّل.»

٣. data_preparation.py — ٧٦٨ سطر

مسؤول عن: التنظيف، ال engineered features، تنظيف النص، ال stoplist، وكل عمليات ال split.

أهم الدوال

السطر	الدالة	بتعمل إيه
197	<code>clean_commit_message_for_nlp()</code>	٩ خطوات regex بترتيب محدد
246	<code>build_stoplist()</code>	بتبني ال token stoplist-693
306	<code>engineer_features()</code>	ال ١١ engineered feature
457	<code>stratified_split()</code>	موجودة للعرض بس — بتوري ال leakage
490	<code>grouped_split()</code>	الأساسية — StratifiedGroupKFold على <code>commit_sha</code>
555	<code>chronological_split()</code>	per-repository، ثانوية
649	<code>split_integrity_report()</code>	دليل آلي إن ال split سليم

سؤال: «إزاي بتنظف رسالة ال commit؟»

«تسع خطوات بترتيب مقصود، في سطر ١٩٧. الترتيب مهم لأن كل خطوة بتعتمد على اللي قبلها:

Co-authored-by headers → dependency bumps → URLs → emails → SHA-40 → PR refs (#1234) →
SHA-8 → version numbers → أسماء الملفات → lowercase → شيل أي حرف مش أبجدي → توحيد
المسافات.

السبب إن ال TF-IDF لو شاف #1234 أو a3f9c2b مش هيتعلم معنى — هيتعلم هوية.»

سؤال: «ال stoplist دي جت مين؟»

«مش مكتوبة بإيدي، اتبنت آلياً في سطر ٢٤٦. بتاخذ كل قيمة في `commit_author` وكل اسم `repository` وتفكّكهم لـ `tokens`، وتضيف عليهم قائمة يدوية لتوقعات البونات (`bors` , `renovate` , `dependabot`) وضوضاء لاحظتها في ال word cloud في Phase 2. الإجمالي ٦٩٣ `token`.
وليه؟ لأنني بصّيت على أهم الكلمات المميّزة قبل التنظيف ولقيت أسماء مؤلفين — `anna` , `kamat` , `yagiz`.
الموديل كان بيتعلّم مين كتب التعديل.»

سؤال: «ليه StratifiedGroupKFold مش GroupShuffleSplit؟» ⚠

ده أخطر سؤال في الملف. الإجابة بالأرقام موجودة في docstring السطر ٤٩٠:

«الأتين بيعملوا `grouping`، بس `GroupShuffleSplit` مبيعلمش `stratification`. جرّبتها وقّست: نسبة الفشل طلعت ٩.٧١٪ في ال `train` مقابل ١٤.٧٢٪ في ال `validation`.
والفرق ده مش تجميلي — هو يفسد اختيار ال `threshold`. `threshold` متحدد على `validation fold` نسبته ١٤.٧٪ مش هينفع على `test fold` نسبته ٩.٧٪، والغلطة اللي هتطلع مش هتبان فرق عن أي أثر `calibration` حقيقي.
فباخذ أول `fold` من `StratifiedGroupKFold` بخمس `folds` — يديني ٢٠٪ `test` مع الحفاظ على التوازن.»

وفي سطر ٥٢٠: فيه `raise ValueError` لو حد مرّر `test_size` غير 0.2 — علشان الطريقة دي مربوطة بال `fold-5` `scheme`.

سؤال: «إزاي أتأكد إن ال split سليم؟»

«مش بطلب من حد يصدّقني. `split_integrity_report()` في سطر ٦٤٩ بتطلّع `results/split_integrity.json` ، وفيه: عدد ال `commits` المشتركة بين أي `partition` (المفروض صفر)، وتوزيع الكلاسات لكل `fold`، وال `temporal invariants` لل `chronological split`. افتح الملف.»

سؤال: «ليه ال chronological split per-repository»؟

«جربت cut زمني عام الأول وطلع غلط. ال run cap-600 بيدي تغطية زمنية مختلفة تماماً لكل مشروع — من ٠.٤ يوم لـ ١٨٢ يوم. فأني cut عام عند ٨٠٪ بيدي نافذة ٩ ساعات على ١١ مشروع من ١٨. فبقطع كل مشروع عند ال quantile بتاعه هو، وال commits اللي بتقع على الحدود بتتشاف بالكامل. وال invariant محقق للـ ١٨ كلهم.»

٤. hybrid_pipeline.py — ٣٧٠ سطر

مسؤول عن: ال ColumnTransformer بأربع فروع، والتلات موديلات.

سؤال: «اشرحلي ال architecture» — السطر ٦٧

```
python ColumnTransformer(transformers=[ ("numerical", StandardScaler(),  
NUMERICAL_FEATURES), ("categorical", OneHotEncoder(handle_unknown="ignore"),  
CATEGORICAL_FEATURES), ("binary", "passthrough", BINARY_FEATURES), ("text",  
TfidfVectorizer(...), TEXT_FEATURE), ], sparse_threshold=0.3, n_jobs=-1)
```

«أربع فروع بتشتغل بالتوازي ويبتدجوا في مصفوفة واحدة. الأرقام بيتعمل لها StandardScaler، التصنيفات OneHotEncoder، ال binary flags بتمرزي ما هي لأنها أصلاً ٠/١، والنص TF-IDF.»

تفاصيل تحفظها:

البارامتر	القيمة	ليه
max_features	3000	سقف على حجم الـ vocabulary
ngram_range	(2, 1)	كلمات فردية وأزواج
min_df	5	الكلمة لازم تظهر في 5 رسائل على الأقل
max_df	0.95	شيل اللي في ٩٥٪ من الرسائل — مالهش قيمة تمييزية
sublinear_tf	True	$1 + \log(tf)$ بدل tf — يقلل أثر التكرار
handle_unknown	"ignore"	مهم ↓
sparse_threshold	0.3	يفضل مصفوفة sparse — الـ TF-IDF بيطلع ~٣٠٠٠ عمود

لو سألوك عن `handle_unknown="ignore"`:

«لو النظام شاف repository أو workflow مشافوش في التدريب، مش بيعق — بيدّي صفر لكل الأعمدة بتاعته. ده شرط لأي نشر حقيقي، لأن مشاريع و workflows جديدة بتظهر طول الوقت.»

سؤال: «ليه `LabelEncoderForBinary` ده؟» — السطر ١٠٧

«الـ labels عندي كلمات: `success` و `failure`. Logistic Regression و Random Forest بيقلوا الكلام. XGBoost بيطلب أرقام.»

فبدل إني أحول الـ labels في المشروع كله — وده كان هيخليني أتوه وأنسى مين رقمه واحد، وفي مسألة غير متوازنة دي غلطة قاتلة — لقيت XGBoost في طبقة رقيقة: بتعمل `fit` لـ `LabelEncoder` واحد عند التدريب، وبتعمل `inverse_transform` عند التوقع.

فالتلات موديلات لهم نفس الواجهة بالطبط، وكود التقييم مستحيل يعاملهم بطريقة مختلفة بالغلط.»

وفيه تفصيلة: الكلاس يعمل expose لـ `named_steps` و `steps` كـ `properties` (سطر ١٣٩ و ١٤٣) علشان الكود اللي بيستخرج الـ `feature importance` يقدر يوصل للـ `pipeline` اللي جوه.

الـ hyperparameters — احفظهم

الموديل	السطر	البارامترات
Logistic Regression	152	<code>solver="liblinear"</code> , <code>class_weight="balanced"</code> , <code>max_iter=1000</code>
Random Forest	170	<code>min_samples_split=10</code> , <code>max_depth=25</code> , <code>n_estimators=200</code> <code>class_weight="balanced"</code> , <code>min_samples_leaf=4</code>
XGBoost	191	<code>learning_rate=0.1</code> , <code>max_depth=8</code> , <code>n_estimators=300</code> <code>reg_alpha=0.1</code> , <code>colsample_bytree=0.85</code> , <code>subsample=0.85</code> <code>tree_method="hist"</code> , <code>scale_pos_weight=...</code> , <code>reg_lambda=1.0</code>

كلهم `random_state=42`.

لو سألوك «إزاي اتعاملت مع الـ `class imbalance` في الموديل نفسه؟»

«تلات طرق حسب المكتبة: `class_weight="balanced"` في LR و RF، و `scale_pos_weight` في XGBoost. التلاتة بيعملوا نفس الحاجة — بيدّوا وزن أكبر للكلاس الأقلية. وده مستقل تماماً عن الـ `threshold` **optimization** — ده بيأثر على التدريب، والـ `threshold` بيأثر على القرار.»

لو سألوك «ليه `liblinear`؟»

«بيشتغل كويس مع مصفوفات `sparse` عالية الأبعاد، ومعايا ~٣٠٩٠ عمود معظمهم TF-IDF.»

السطر	الدالة
237	<code>build_text_only_preprocessor()</code>
255	<code>build_structured_only_preprocessor()</code>
272	<code>build_categorical_only_preprocessor()</code> ← اللي كسب

cross_validation.py .0 — ۲۳۶ سطر

مسؤول عن: بروتوكول التقييم كله. أهم ملف في المشروع من ناحية المنهج.

run_grouped_cv() — السطر ٩٦

الحلقة بتعمل الآتي لكل fold:

```
outer = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42) كل fold:
train_pool, test = ال outer split inner = StratifiedGroupKFold(n_splits=5) ال
train_pool موديل جديد تماماً كل inner split pipeline = factory() بس fit_df, val_df =
fold pipeline.fit(fit_df) threshold = test test_proba = val_df اختبار على
predict(test_df) oof_proba[test_idx] = test_proba
```

⚠ أربع تفاصيل هنا كل واحدة فيهم سؤال

١. ليه `pipeline_factories` مش `pipelines` جاهزة؟ (السطر ٩٦)

«بمَرِّر دوال بترجّع pipeline جديد، مش instance. لو مَرِّرت instance واحد، ال folds الخمسة هيتدربوا على نفس الكائن ويتراكموا فوق بعض. ال factory بتضمن إن كل fold بيدرب موديل مستقل تماماً.»

٢. ال inner split (السطر ~١٢٨)

«جَوّه كل fold بعمل split ثاني — كمان grouped على ال commit — علشان أختار ال threshold. فال threshold متشافش ال test fold ولا مرة. ودي كانت الغلطة اللي كلّفتني ٠.١٢٦.»

٣. ال assert (السطر ١٩٠)

```
python assert not np.isnan(oof_proba).any(), "every row must receive one out-of-fold prediction"
```

«ده ضمان إن ال ٩٧٧٢ صف كلهم أخذوا توقع واحد بالطبط، من موديل متدريش على أي run من ال commit بتاعهم. لو fold سقط صف، البرنامج بيوقع بدل ما يطلع رقم غلط.»

٤. ليه decision metrics per-fold و ranking metrics pooled؟

«ال ROC-AUC و PR-AUC بيتحسب مرة واحدة على ال ٩٧٧٢ توقع المجمعين — لأنهم threshold-independent فمفيش معنى لمتوسطهم. أما F1 و precision و recall فبيتحسب لكل fold لوحده ويقول المتوسط $\pm$ الانحراف المعياري — لأن كل fold ليه threshold مختلف. ودي السبب إن عندي roc_auc و roc_auc_per_fold الاتنين في ال JSON — الأول pooled والثاني متوسط.»

وكان بيسجل دليل في كل fold

```
python "commit_overlap_fit_test": ... ← المفروض صفر ← "commit_overlap_val_test": ... ← المفروض صفر
```

«مش بفترض إن ال grouping شغال — بقيسه في كل fold ويكتبه في ال output.»

٦. train_evaluate.py — ٥١٠ سطر

مسؤول عن: المقاييس، ال ablation، والنموذج المالي.

السطر	الدالة
88	<code>compute_binary_metrics()</code>
162	<code>get_proba_and_classes()</code> — يتعامل مع الثلاث pipelines بنفس الطريقة
221	<code>train_all_models()</code>
274	<code>run_ablation_study()</code>
337	<code>compute_business_metrics()</code>
420	<code>identify_best_model()</code>

سؤال: «إزاي حسبت ال \$243,670؟» — السطر ٣٣٧

النموذج:

البند	القيمة	من إيه
compute مهدور لكل فشل	\$0.064	٨ دقائق × \$0.008/دقيقة (سعر GitHub المعلن)
context-switch لكل فشل	\$18.75	١٥ دقيقة × \$75/ساعة
تكلفة الإنذار الكاذب	\$2.50	دقيقتين × \$75/triage ساعة
نسبة الفشل	10.97%	المقيسة، مش مفترضة

```
caught = pipelines_per_day × failure_rate × recall
false_alarms = caught × (1 - precision)
/ precision
net = caught × (0.064 + 18.75) - false_alarms × 2.50
```

⚠ أهم جملة تقولها عن الملف ده:

«النسخة القديمة كان فيها ثلاث عيوب كلهم بيرفعوا الرقم: كانت بتفترض نسبة فشل ٣٠٪ بدل ١١٪ المقيسة، وكانت بتسعر الفائدة كدقائق triage موقرة بدل تكلفة الـ compute والـ context-switch، ومكانتش بتخصم تكلفة الإنذار الكاذب خالص.

والعيب الثالث ده خلّى التقدير دالة في الـ recall وحده. وده اللي فسّر نتيجة غريبة كنت مستغريها: الموديل المضبوط كان بيوقّر أقل من الموديل غير المضبوط — لأنه باع precision علشان الـ recall، والـ recall بس هو اللي كان بيتسعر.

دلوقتي الـ precision داخل في المعادلة، فالموديل مش قادر يشتري رقم أحسن إنه يفلج أكثر.»

٧. `run_corrected_evaluation.py` — ٩٧١ سطر

مسؤول عن: المشغل الرئيسي. كل رقم وكل `figure` في فصل ٧ بيطلع من هنا.

السطر	الدالة	بتطّلع
104	<code>build_attribution_ladder()</code>	سلم النسب (٠.٠١٥ / ٠.١٢٦ / ٠.٠٦٠)
215	<code>plot_cv_metrics()</code>	Figure 14
270	<code>plot_ablation()</code>	Figure 15
323	<code>plot_attribution_waterfall()</code>	Figure 18
392	<code>plot_business_sensitivity()</code>	Figure 19
448	<code>plot_oof_curves()</code>	Figures 13 و 22
508	<code>plot_oof_confusion()</code>	Figure 12
551	<code>plot_threshold_curves()</code>	Figure 20
632	<code>plot_before_after_threshold()</code>	Figure 21
715	<code>evaluate_chronological()</code>	التقييم الزمني

سؤال: «إزاي عرفت إن ال leakage كلفت ٠.٠٦٠ بالظبط؟» — السطر ١٠٤

» `build_attribution_ladder()` بتعمل تفكيك بخطوة واحدة كل مرة. بتبدأ من الإعداد الأصلي بالظبط — stratified split و threshold على ال test — وتتغير حاجة واحدة بس في كل درجة:

A. الإعداد الأصلي → 0.5924 B. + شيل ال C -0.060 → commit leakage + اختار ال threshold على validation
D -0.126 → . + متوسط على خمس +0.015 folds

كل درجة نفس الداتا ونفس ال hyperparameters ونفس ال seed. الحاجة الوحيدة اللي بتتغير هي البروتوكول. علشان كده أنا بقيس التكلفة مش بقدرها.»

٨. أسئلة الكود المتوقعة — إجابات سريعة

السؤال	الإجابة
«فين ال splits؟»	<code>data_preparation.py</code> ، سطر ٤٩٠ للأساسية
«فين ال threshold بيتحدد؟»	<code>cross_validation.py</code> جّوه ال inner split — أبدأً مش على <code>test</code>
«فين الأرقام المنشورة؟»	<code>results/*.json</code> . الرسالة بتقتبس منهم مش العكس
«إزاي أعيد النتائج؟»	<code>./reproduce.sh</code> — ٣ دقائق من checkout نضيف
«ال random seed؟»	42 في كل مكان
«إصدارات المكتبات؟»	متبّنة في <code>requirements.txt</code> — <code>scikit-learn 1.8.0</code> , <code>xgboost 3.2.0</code>
«فين الموديل المدرّب؟»	<code>models/best_tuned_threshold_xgb.joblib</code> + <code>metadata sidecar</code>
«كام سطر كود؟»	٣٤٠٥ سطر في ال ٦ modules الأساسية

٩. لو سألوك حاجة مش عارفها

ما تخترعش. أبدأً. أسوأ حاجة تعملها إنك تقول رقم غلط بثقة.

«مش فاكّر التفاصيل دي بالضبط دلوقتي، ومش عايز أقول حاجة غلط. الملف موجود معايا وأقدر أفتحه وأوريك السطر. الفكرة العامة إن...»

وبعدين قول اللي إنت متأكد منه.

وال fallback اللي دايماً صح:

«كل رقم في الرسالة يتولّد من `results/*.json` ، وكل ملف منهم يتولّد بأمر واحد. مفيش رقم مكتوب بإيدي.»

١٠. قبل ما تدخل — احفظ التلاتة دول بس

1. `data_preparation.py` سطر ٤٩٠ — ال `grouped split`، والسبب (٩٠.٧١٪ مقابل ١٤.٧٢٪)

2. `cross_validation.py` سطر ٩٦ — ال `inner split` اللي بيختار ال `threshold`

3. `run_corrected_evaluation.py` سطر ١٠٤ — سلم النسب

التلات نقط دول هما المنهج كله. أي سؤال كود ثاني تقدر تجاوبه بـ«الملف معايب، أوريك السطر».